

Higher-Order Metaphysics

Week Four: The Curry-Howard correspondence and some extended type systems

CIAN DORR
(New York University: cian.dorr@nyu.edu)
4th October 2023

1 A quick introduction to Classicism

Many widely-studied logical systems are closed under *substitution of logical equivalents*: if formulae P and Q are logically equivalent, then we freely substitute each for the other. Neither H nor H_0 has this feature. For example, although $p \vdash \neg\neg p$ and $\neg\neg p \vdash p$, and $\vdash p =_t p$, $\nmid p =_t \neg\neg p$.

Bacon and Dorr use the name ‘Classicism’ for the minimal strengthening of H that does have this property.¹ In a natural deduction formalism, this corresponds to adding the following new rule:

$$\frac{P \vdash Q \quad Q \vdash P \quad \Gamma \vdash R[P/v]}{\Gamma \vdash R[Q/v]} \text{ Subst}$$

¹ Actually they are working in a different type system where e can’t appear on the right of $\rightarrow$; I’ll gloss over this today.

There are many other ways of axiomatizing this theory. Another uses a “rule of equivalence”:

$$\frac{\vdash P \leftrightarrow Q}{\vdash (\lambda \vec{v}.P) = (\lambda \vec{v}.Q)} \text{ RE}$$

A third, especially economical, way just adds a certain group of closed identities as axioms:

Commutativity	$(\lambda p q. p \wedge q) =_{t \rightarrow t \rightarrow t} (\lambda p q. q \wedge p)$
Absorption	$(\lambda p q. p \vee (p \wedge q)) =_{t \rightarrow t \rightarrow t} (\lambda p q. p)$
Distribution	$(\lambda p q r. p \vee (q \wedge r)) =_{t \rightarrow t \rightarrow t} (\lambda p q r. (p \vee q) \wedge (p \vee r))$
Complementation	$(\lambda p q. p \wedge \neg p) =_{t \rightarrow t \rightarrow t} (\lambda p q. q \wedge \neg q)$
Involution	$(\lambda p. \neg \neg p) =_{t \rightarrow t} (\lambda p. p)$
$\forall$-Absorption	$(\lambda X y. X y \vee \forall_{\sigma} X) =_{(\sigma \rightarrow t) \rightarrow \sigma \rightarrow t} (\lambda X y. X y)$
$\forall$-Distribution	$(\lambda X p. p \vee \forall_{\sigma} X) =_{(\sigma \rightarrow t) \rightarrow t \rightarrow t} (\lambda X p. \forall y^{\sigma}. p \vee X y)$
Self-identity	$(\lambda x. x =_{\sigma} x) =_{\sigma \rightarrow t} (\lambda x. \top)$
Leibniz	$(\lambda Z x y. x =_{\sigma} y \wedge Z x) =_{(\sigma \rightarrow t) \rightarrow \sigma \rightarrow \sigma \rightarrow t} (\lambda Z x y. x =_{\sigma} y \wedge Z y)$

This is all very metaphysically controversial!

2 Extensionalism

Much stronger than Classicism is ‘Extensionalism’, which may be axiomatised by adding the following axiom-scheme to H —or equivalently, to C or to H_0 :

$$\text{Extensionality} \quad (\forall \vec{v}. F\vec{v} \leftrightarrow G\vec{v}) \rightarrow F =_{\vec{\sigma} \rightarrow t} G$$

This seems false to me—indeed, so obviously false as to prompt serious questions whether people who seem to endorse it are understanding the language in the same way as me.

3 Environment-relative higher-order syntax

So far we have used a higher order syntax where each variable has a built-in type, so that the question ‘Is this a term and if so what is its type?’ has a once and for all answer. As I mentioned, one can also set things up such that variables lack built-in types, and terms have a type only relative to an “environment” that assigns types to their free variables. To facilitate interesting later developments we will take these environments to be *finite ordered lists* of the form $v_1 : \sigma_1, \dots, v_n : \sigma_n$, where $v_1, \dots, v_n$ are *distinct* variables and $\sigma_1, \dots, \sigma_n$ are types.²

Given a signature Σ (typed collection of constants), we now define $\mathcal{L}(\Sigma)$ as a set of “typing sequents”, each of which is a triple $\langle \Gamma, A, \sigma \rangle$ of an environment Γ , a string A , and a type σ . We read $\langle \Gamma, A, \sigma \rangle \in \mathcal{L}(\Sigma)$ as ‘ A is a term with type σ relative to environment Γ ’, and write it as $\Gamma \vdash_{\mathcal{L}(\Sigma)} A : \sigma$. We can define it as the smallest set of triples $\vdash$ meeting the following conditions:

$$\begin{array}{c} \frac{\sigma \in \mathbf{Types}}{\Gamma, v : \sigma \vdash v : \sigma} \text{Var} \quad \frac{\Gamma \vdash A : \sigma \quad \tau \in \mathbf{Types}}{\Gamma, v : \tau \vdash A : \sigma} \text{AddVar} \quad \frac{c :_{\Sigma} \sigma}{\vdash c :_{\Sigma} \sigma} \text{Const} \\[10pt] \frac{\Gamma, v : \sigma \vdash A : \tau}{\Gamma \vdash (\lambda v^{\sigma}. A) : \sigma \rightarrow \tau} \text{Abs} \quad \frac{\Gamma \vdash A : \sigma \rightarrow \tau \quad \Gamma \vdash B : \sigma}{(\lambda v^{\sigma}. A) B : \tau} \text{App} \end{array}$$

‘ $\Gamma, v : \sigma'$ ’ means ‘the environment derived from Γ by adding ‘ $v : \sigma'$ ’ to the end, assuming v does not already occur in Γ ’; when v does occur in Γ this is undefined and the relevant rule does not apply. There are many many other ways of defining the same relation.³

The following can all be proved by straightforward inductions:

- Unique typing: if $\Gamma \vdash A : \sigma$ and $\Gamma \vdash A : \sigma'$ then $\sigma = \sigma'$.
- Permutation and Weakening: if $\Gamma \vdash A : \sigma$ and Γ' contains $v : \sigma$ whenever Γ does, then $\Gamma' \vdash A : \sigma$.

² In the standard system, we’ll be able to prove that the order of this list doesn’t matter (so we could just as well take it to be an unordered set of pairs); however we give it an order to facilitate comparison with some non-standard systems we’ll discuss later (ones that generate fewer or more terms).

³ For another definition (more complex but more suggestive of interesting weakenings) see Bacon, PIHOL, Table 10.1 (this uses Curry rather than Church typing but could be straightforwardly adapted).

- Substitution: if $\Gamma, v : \sigma \vdash A : \tau$ and $\Gamma \vdash B : \sigma$, then $\Gamma \vdash A[B/v] : \tau$.

A variant of this system due to Curry omits the type annotation on the lambda terms (for which I use a superscript). Now the unique typing property fails: even a closed term (for example, $(\lambda x.p)(\lambda y.y)$) can admit multiple types in a single environment.

4 The correspondence between types and formulae

Implicational logic is propositional logic with $\rightarrow$ as the only connective. Let's identify logics with their "provability" relations understood as sets of pairs ("sequents") $\langle \Gamma, P \rangle$, where P is a formula and Γ is a "context": a finite ordered list of formulae.⁴ Then the *intuitionistic* implicational logic for a certain propositional language $\mathcal{P}$ (specified by a set of sentence-constants) can be identified with the smallest such relation $\vdash_I$ such that:

$$\begin{array}{c} \frac{P \in \mathcal{P}}{\Gamma, P \vdash P} \text{ Assumption} \qquad \frac{\Gamma \vdash P \quad Q \in \mathcal{P}}{\Gamma, Q \vdash P} \text{ Weakening} \\[10pt] \frac{\Gamma, P \vdash Q}{\Gamma \vdash P \rightarrow Q} \rightarrow I \qquad \frac{\Gamma \vdash P \rightarrow Q \quad \Gamma \vdash P}{\Gamma \vdash Q} \rightarrow E \end{array}$$

If we take $\mathcal{P}$ to be Types (i.e. the propositional language with just 'e' and 't' as sentence-constants), it is immediate that $\Gamma \vdash_{L(\emptyset)} A : \sigma$ iff $\Gamma^- \vdash_I \sigma$, where Γ^- is the result of replacing each pair $v : \tau$ in Γ with just τ .⁵

How is this useful? One application is showing that we can't build terms in certain types. We know that everything provable in intuitionistic logic is a tautology (though the reverse is not true), so we can show that a formula is not provable in intuitionistic logic by showing that it has an F on some line of its truth table. This lets us show, e.g., that there is no combinator of type t , of type $(t \rightarrow t) \rightarrow t$, etc.

5 The correspondence between terms and proofs

When we're defining logics it is sometimes useful to also define a notion of *proof* (aka *derivation*) where proofs are syntactic objects.⁶

In the case of natural deduction, the most useful notion of proof for metalogical purposes (where efficiency is not an issue) identifies proofs with *trees*.⁷ We need to put *labels*⁸ on the leaves of our trees, since the $\rightarrow$ Intro rule discharges assumptions by their label. So what we define is the relation " Π is a proof of P from labelled context Γ ", where a labelled context is a finite list $\langle v_1 : Q_1, \dots, v_n : Q_n \rangle$ of pairs of a label v_i and a

⁴ In the standard systems we'll be able to prove that order and repetitions don't matter, but using lists facilitates comparison with nonstandard systems.

⁵ We can also consider extensions of $\vdash_I$ with certain *axioms*,

⁶ One reason: in very many cases, the question 'is this formula/sequent in the logic' will not be *decidable*, whereas if we set up the notion of 'proof' properly, the question 'is this thing a proof of this formula/sequent' will be decidable. Another reason: once we have a notion of proof we can define special "normal" proofs, show that for every proof there is a normal proof of the same thing, and use this to establish negative facts about what isn't provable.

⁷ So we don't have to make arbitrary choices about order.

⁸ Traditionally these are numbers, but it doesn't matter what they are.

formula Q_i where no two v_i are identical. We write this as $\Gamma \vdash \Pi : P$. It can be defined as the smallest such relation meeting the following conditions:⁹

⁹I use dashed lines for the object-language proofs and the solid lines, as usual, to abbreviate ‘if...then’ sentences.

$$\begin{array}{c}
 \frac{P \in \mathcal{P}}{\Gamma, v : P \vdash (P)^v : P} \text{Assumption} \qquad \frac{\Gamma \vdash \Pi : P \quad Q \in \mathcal{P}}{\Gamma, v : Q \vdash \Pi : P} \text{Weakening} \\
 \\
 \frac{\Gamma, v : P \vdash \Pi : Q}{\Gamma \vdash \boxed{\begin{array}{c} \Pi \\ \hline P \rightarrow Q \end{array}} \rightarrow I v : P \rightarrow Q} \rightarrow I \qquad \frac{\Gamma \vdash \Pi : P \rightarrow Q \quad \Gamma \vdash \Delta : P}{\Gamma \vdash \boxed{\begin{array}{c} \Pi \quad \Delta \\ \hline Q \end{array}} \rightarrow E : Q} \rightarrow E
 \end{array}$$

Two obvious inductions show that $\Gamma \vdash P$ iff for any labeled context Γ^+ that gives labels $v_1, \dots, v_n$ to the elements $Q_1, \dots, Q_n$ of Γ , there is some proof Π such that $\Gamma^+ \vdash \Pi : P$.

This will hopefully make the other half of the Curry-Howard correspondence—between terms and proofs—obvious.

6 Simplifying proofs, reducing terms

The counterparts of α -, β -, η -, and $\beta\eta$ -equivalence under the isomorphism will be equivalence relations on *proofs*. Such relations have been studied extensively by proof-theorists like Gentzen and Prawitz, especially with a view to the “simplification” or “reduction” of proofs. Corresponding to single steps of β - and η -reduction are the following simplification operations on proofs:

$$\begin{array}{ccc}
 \begin{array}{c} \Pi \\ \vdots \\ (P)^v \quad Q \\ \hline P \rightarrow Q \end{array} \rightarrow I v \quad \begin{array}{c} \Delta \\ \vdots \\ P \end{array} \rightarrow E \\
 \hline Q \\ \vdots \\ \Sigma
 \end{array} \rightsquigarrow_{\beta} \begin{array}{c} \Pi[\Delta/(P)^v] \\ \vdots \\ Q \\ \vdots \\ \Sigma \end{array} \qquad \begin{array}{c} \Pi \\ \vdots \\ P \rightarrow Q \quad (P)^v \\ \hline Q \end{array} \rightarrow E \quad \rightsquigarrow_{\eta} \quad \begin{array}{c} \Pi \\ \vdots \\ P \rightarrow Q \\ \vdots \\ \Sigma \end{array}
 \end{array}$$

Corresponding to the notion of a term in β -normal or $\beta\eta$ -normal form is that of a “normal” proof, one free of “detours” (Prawitz) or “hills” (Gentzen). They both proved (though Gentzen never published his) “normalization” theorems according to which every proof can be turned into one in normal form (incapable of further reduction), and the normal forms in any equivalence class are all α -equivalent.

7 Extended types

One natural thought at this point is: what about a language with new ways of forming types, corresponding to the other standard connectives of propositional logic?

Suppose that instead of our simple type system $\mathcal{T}$ we use the *extended type system* $\mathcal{T}^+$, defined as the smallest set that contains e , t , $\perp$, and $\top$, and such that it contains $(\sigma \rightarrow \tau)$, $(\sigma \wedge \tau)$, and $(\sigma \vee \tau)$ whenever it contains σ and τ .¹⁰

To do anything interesting with these new types, we will need new *term-forming* operations which stand to the new type-forming operations as our existing operations (abstraction and application) stand to $\rightarrow$.

Here, the CH correspondence is heuristically useful: we'd expect our new term-forming operations to correspond to *proof*-forming operations in propositional logic with $\wedge$, $\vee$, $\top$, and $\perp$. Here are the standard intuitionistic rules for these connectives, with explicit proofs.¹¹

$$\begin{array}{c}
 \frac{\Gamma \vdash \Pi : P \quad \Gamma \vdash \Delta : Q}{\Gamma \vdash \boxed{\begin{array}{c} \Pi \quad \Delta \\ \hline P \wedge Q \end{array}} : P \wedge Q} \wedge I \qquad \frac{\Gamma \vdash \Pi : P \wedge Q}{\Gamma \vdash \boxed{\begin{array}{c} \Pi \\ \hline P \end{array}} : P} \wedge E1 \qquad \frac{\Gamma \vdash \Pi : P \wedge Q}{\Gamma \vdash \boxed{\begin{array}{c} \Pi \\ \hline Q \end{array}} : Q} \wedge E2 \\
 \\
 \frac{\Gamma \vdash \Pi : P \quad Q \in \mathcal{P}}{\Gamma \vdash \boxed{\begin{array}{c} \Pi \\ \hline P \vee Q \end{array}} : P \vee Q} \vee I1 \qquad \frac{\Gamma \vdash \Pi : Q \quad P \in \mathcal{P}}{\Gamma \vdash \boxed{\begin{array}{c} \Pi \\ \hline P \vee Q \end{array}} : P \vee Q} \vee I2 \\
 \\
 \frac{\Gamma \vdash \Pi : P \vee Q \quad \Gamma, u : P \vdash \Delta_1 : R \quad \Gamma, v : Q \vdash \Delta_2 : R}{\Gamma \vdash \boxed{\begin{array}{c} \Pi \quad \Delta_1 \quad \Delta_2 \\ \hline R \end{array}} : R} \vee E \\
 \\
 \frac{}{\Gamma \vdash \boxed{\begin{array}{c} \top \\ \hline \top \end{array}} : \top} \top I \qquad \frac{\Gamma \vdash \Pi : \perp \quad P \in \mathcal{P}}{\Gamma \vdash \boxed{\begin{array}{c} \Pi \\ \hline P \end{array}} : P} \perp E
 \end{array}$$

By following the CH correspondence, we can see what new *term-forming operations* would stand to these type-forming operations as the operations of application and abstraction stand to $\rightarrow$.¹²

¹⁰ A common alternative notation uses 0, 1, + and $\times$ instead of $\perp$, $\top$, $\vee$ and $\wedge$.

¹¹ In this setting it is standard to define $\neg P$ as $P \rightarrow \perp$. By contrast, the classically-acceptable definitions of $\wedge$ and $\vee$ in terms of $\rightarrow$ and $\neg$ don't work without DNE.

¹² Since we normally write terms as strings rather than trees we replace proof annotations with greek letters, just as we replaced " $\rightarrow$ " with the letter λ .

$$\begin{array}{c}
\frac{\Gamma \vdash A : \sigma \quad \Gamma \vdash B : \tau}{\Gamma \vdash \langle A, B \rangle : \sigma \wedge \tau} \wedge I \quad \frac{\Gamma \vdash A : \sigma \wedge \tau}{\Gamma \vdash \pi_1 A : \sigma} \wedge E1 \quad \frac{\Gamma \vdash A : \sigma \wedge \tau}{\Gamma \vdash \pi_2 A : \tau} \wedge E2 \\
\\
\frac{\Gamma \vdash A : \sigma \quad \tau \in \mathcal{T}}{\Gamma \vdash \iota_\tau^1 A : \sigma \vee \tau} \vee I1 \quad \frac{\Gamma \vdash B : \tau \quad \sigma \in \mathcal{T}}{\Gamma \vdash \iota_\sigma^2 B : \sigma \vee \tau} \vee I2 \\
\\
\frac{\Gamma \vdash A : \sigma \vee \tau \quad \Gamma, u : \sigma \vdash B : \rho \quad \Gamma, v : \tau \vdash C : \rho}{\Gamma \vdash \delta A(u.B)(v.C) : \rho} \vee E \\
\\
\frac{}{\Gamma \vdash * : \top} \top I \quad \frac{\Gamma \vdash A : \perp \quad \sigma \in \mathcal{T}}{\Gamma \vdash \varepsilon_\sigma A : \sigma} \perp E
\end{array}$$

8 β - and η -equivalence and reduction for extended types

To do any interesting *logical* work with our new terms we will need some rules for manipulating them, analogous to β - and η -conversion. These rules will correspond to rules of *proof simplification*: the β -rules correspond to getting rid of an introduction rule followed immediately by the corresponding elimination rule, and the η -rules are the other way around.

Just like most syntactic notions, the relevant equivalence relations $\sim_\beta$, $\sim_{\eta'}$, $\sim_{\beta\eta}$ need to be relativized to a context Γ —we write $\Gamma \vdash A \sim B$.¹³

$$\begin{array}{c}
\frac{\Gamma \vdash A : \sigma \quad \Gamma \vdash B : \tau}{\Gamma \vdash \pi^1 \langle A, B \rangle \sim A} \beta_\wedge^1 \quad \frac{\Gamma \vdash A : \sigma \quad \Gamma \vdash B : \tau}{\Gamma \vdash \pi^2 \langle A, B \rangle \sim B} \beta_\wedge^2 \\
\\
\frac{\Gamma \vdash A : \sigma \wedge \tau}{\Gamma \vdash \langle \pi^1 A, \pi^2 A \rangle \sim A} \eta_\wedge \quad \frac{\Gamma \vdash A : \top}{\Gamma \vdash * \sim A} \eta_* \\
\\
\frac{\Gamma \vdash A : \sigma \quad \Gamma \vdash B : \tau \quad \Gamma \vdash C : \tau}{\Gamma \vdash \delta(\iota_\tau^1 A)(u.B)(v.C) \equiv B[A/u]} \beta_\vee^1 \quad \frac{\Gamma \vdash A : \sigma \quad \Gamma \vdash B : \tau \quad \Gamma \vdash C : \tau}{\Gamma \vdash \delta(\iota_\sigma^2 A)(u.B)(v.C) \equiv C[A/v]} \beta_\vee^2 \\
\\
\frac{\Gamma \vdash A : \sigma \vee \tau \quad \Gamma \vdash B : \rho}{\Gamma \vdash \delta A(u.B[\iota_\tau^1 u/x])(v.B[\iota_\sigma^2 v/x]) \sim B[A/x]} \eta_\vee \quad \frac{\Gamma \vdash A : \perp \quad \Gamma \vdash B : \sigma}{\Gamma \vdash \varepsilon_\sigma A \sim B} \eta_\perp
\end{array}$$

¹³ In the case of the simple type system, such relativization turns out not to be needed—terms that are equivalent relative to one context are equivalent relative to any context in which both are typeable terms. But in the extended system the relativization is crucial: if it were dropped, the rules for $*$ and ε would reduce $\sim$ to triviality.

9 Normal forms for extended types

In the setting of implicational logic [simple type theory], a crucial feature of proofs [terms] in normal form is the *subformula* [*subtype*] property: the conclusion [type] of every subproof [subterm] is either a subformula [subtype] of the conclusion [type] of the proof [term], a subformula [subtype] of one of the assumptions [types of the variables], or a subformula [subtype] of an axiom [type of a constant].

Prawitz and Gentzen showed (independently) that we can define a notion of normal form for the extended language/type system that has the same property.

This means that if we don't have any new *constants* in the extended types, there is a sense in which we don't get anything new by introducing the extended types: every *sentence* (in fact, every simply-typed closed term) in the extended language will be $\sim_{\beta\eta}$ -equivalent to [hence synonymymous with?]something in the old language.

If we *do* have constants floating around in the new types, we can still get a sort of translation from the extended language to an old language by systematically replacing each such constant with an expression built up from only simply-typed constants.

10 Logics for extended types